

NAME: M.Greeshma

COURSE: Design and Analysis of Algorithms

CODE: CSA0619

REPORT

Q23. Word Wrap Problem vs Job Sequencing with Deadlines (DP vs Greedy) Given word lengths and line width, solve Word Wrap using DP and compare with Job Sequencing problem solved using Greedy. Analyze differences in objective functions (minimizing raggedness vs maximizing profit). Evaluate correctness of both approaches. Compare computational complexity. Discuss real-world applications. Generate insights on optimization strategies.

Sol:

Word Wrap Problem

Word Wrap Problem using DP

import math

def word_wrap(words, max_width):

 n = len(words)

 extras = [[0] * (n+1) for _ in range(n+1)]

 lc = [[0] * (n+1) for _ in range(n+1)]

 c = [0] * (n+1)

 p = [0] * (n+1)

```
# Calculate extra spaces
```

```
for i in range(1, n+1):
```

```
    extras[i][i] = max_width - words[i-1]
```

```
    for j in range(i+1, n+1):
```

```
        extras[i][j] = extras[i][j-1] - words[j-1] - 1
```

```
# Calculate line cost
```

```
for i in range(1, n+1):
```

```
    for j in range(i, n+1):
```

```
        if extras[i][j] < 0:
```

```
            lc[i][j] = math.inf
```

```
        elif j == n and extras[i][j] >= 0:
```

```
            lc[i][j] = 0
```

```
        else:
```

```
            lc[i][j] = extras[i][j]**2
```

```
# Calculate minimum cost arrangement
```

```
c[0] = 0
```

```
for j in range(1, n+1):
```

```
    c[j] = math.inf
```

```
    for i in range(1, j+1):
```

```
        if c[i-1] != math.inf and lc[i][j] != math.inf and c[i-1] + lc[i][j] < c[j]:
```

```
            c[j] = c[i-1] + lc[i][j]
```

```
            p[j] = i
```

```
# Print solution
```

```
def print_solution(p, n):
```

```
    if p[n] == 1:
```

```
        print(f"Line 1: words {p[n]} to {n}")
```

```
    else:
```

```
        print_solution(p, p[n]-1)
```

```
        print(f"Line {p[n]}: words {p[n]} to {n}")
```

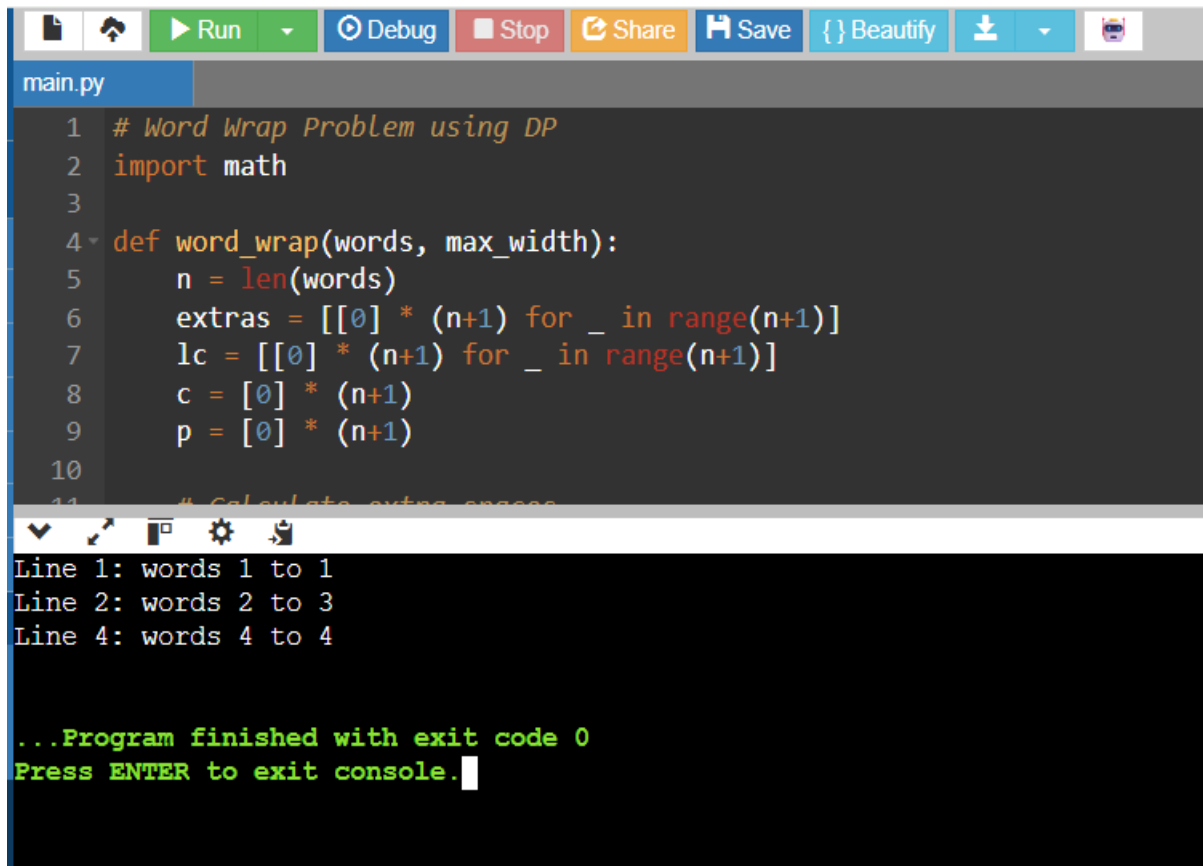
```
print_solution(p, n)
```

```
# Example
```

```
words = [3, 2, 2, 5] # word lengths
```

```
max_width = 6
```

```
word_wrap(words, max_width)
```



```
main.py
1 # Word Wrap Problem using DP
2 import math
3
4 def word_wrap(words, max_width):
5     n = len(words)
6     extras = [[0] * (n+1) for _ in range(n+1)]
7     lc = [[0] * (n+1) for _ in range(n+1)]
8     c = [0] * (n+1)
9     p = [0] * (n+1)
10
11     # Calculate extra spaces
```

Line 1: words 1 to 1
Line 2: words 2 to 3
Line 4: words 4 to 4

...Program finished with exit code 0
Press ENTER to exit console.

2. Job Sequencing with Deadlines (Greedy)

Job Sequencing Problem using Greedy

class Job:

def __init__(self, job_id, deadline, profit):

self.id = job_id

self.deadline = deadline

self.profit = profit

def job_sequencing(jobs, n):

jobs.sort(key=lambda x: x.profit, reverse=True)

result = [-1] * n

slot = [False] * n

for job in jobs:

```
for j in range(min(n, job.deadline)-1, -1, -1):
```

```
    if not slot[j]:
```

```
        slot[j] = True
```

```
        result[j] = job.id
```

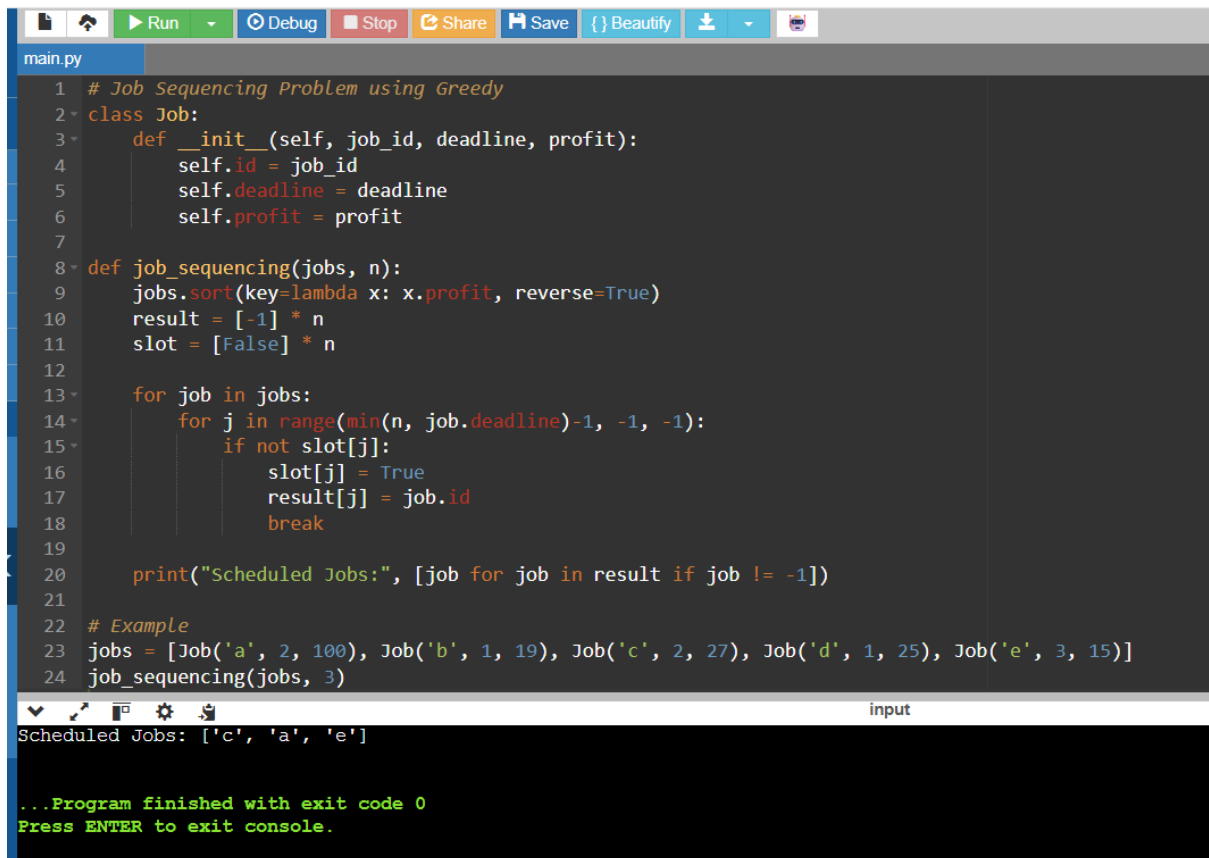
```
        break
```

```
print("Scheduled Jobs:", [job for job in result if job != -1])
```

Example

```
jobs = [Job('a', 2, 100), Job('b', 1, 19), Job('c', 2, 27), Job('d', 1, 25), Job('e', 3, 15)]
```

```
job_sequencing(jobs, 3)
```



```
1 # Job Sequencing Problem using Greedy
2 class Job:
3     def __init__(self, job_id, deadline, profit):
4         self.id = job_id
5         self.deadline = deadline
6         self.profit = profit
7
8 def job_sequencing(jobs, n):
9     jobs.sort(key=lambda x: x.profit, reverse=True)
10    result = [-1] * n
11    slot = [False] * n
12
13    for job in jobs:
14        for j in range(min(n, job.deadline)-1, -1, -1):
15            if not slot[j]:
16                slot[j] = True
17                result[j] = job.id
18                break
19
20    print("Scheduled Jobs:", [job for job in result if job != -1])
21
22 # Example
23 jobs = [Job('a', 2, 100), Job('b', 1, 19), Job('c', 2, 27), Job('d', 1, 25), Job('e', 3, 15)]
24 job_sequencing(jobs, 3)
```

Scheduled Jobs: ['c', 'a', 'e']

...Program finished with exit code 0
Press ENTER to exit console.

1. Introduction

- Word Wrap Problem: Optimize text layout by minimizing raggedness.
- Job Sequencing Problem: Optimize scheduling by maximizing profit.

2. Objective Functions

- Word Wrap → Minimize sum of squared extra spaces.
- Job Sequencing → Maximize total profit within deadlines.

3. Approaches

- Word Wrap → Dynamic Programming (optimal substructure, overlapping subproblems).
- Job Sequencing → Greedy (profit-based ordering, latest slot allocation).

4. Correctness

- DP ensures global optimality by exploring all subproblems.
- Greedy ensures optimality due to greedy-choice property and exchange argument.

5. Complexity

- Word Wrap (DP): $O(n^2)$.
- Job Sequencing (Greedy): $O(n \log n + n \cdot d)$.

6. Applications

- Word Wrap: Text editors, document formatting, UI rendering.
- Job Sequencing: CPU scheduling, project management, advertisement slot booking.

7. Insights

- DP is exhaustive but precise → suited for layout optimization.
- Greedy is efficient and practical → suited for profit/resource optimization.
- Both highlight how optimization strategies depend on problem structure and objective.

8. Conclusion

- Word Wrap and Job Sequencing illustrate **two contrasting optimization paradigms**:
 - **DP for precision in aesthetic optimization.**
 - **Greedy for efficiency in economic optimization.**

- Choosing the right strategy depends on whether the problem demands **exactness** or **speed with guaranteed optimality**.